



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

6. 计算机视觉中的深度学习 (二)

张宏志

机器学习研究中心, 计算学部
2026 春

zhanghz@hit.edu.cn 390684271@qq.com



本节内容

- 深度学习基础
 - 深度学习基本介绍
 - 人工神经网络基础
 - 经典网络结构
- 视觉神经网络与典型训练范式
 - 视觉Transformer
 - 神经网络的典型训练范式
 - 典型深度学习框架

规格严格 功夫到家

本节内容 - 视觉神经网络与典型训练范式

➤ 视觉Transformer

➤ 神经网络的典型训练范式

➤ 典型深度学习框架

规格严格 功夫到家

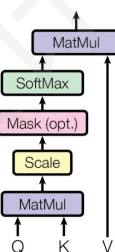
Transformer

➤ 自然语言 → 计算机视觉 → 多模态

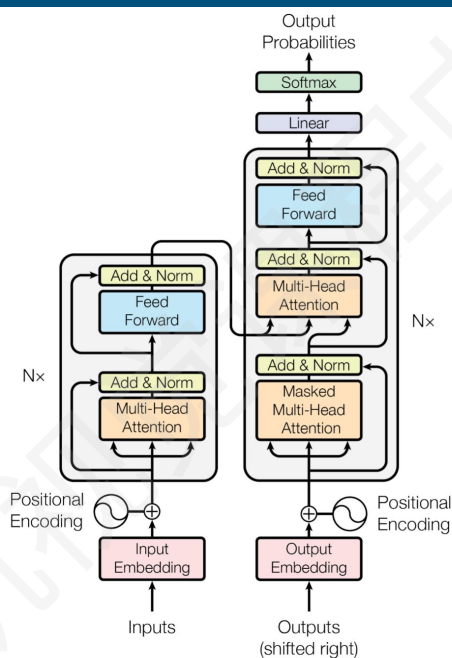
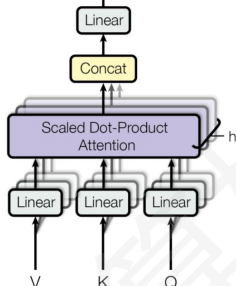
➤ 编码器-解码器结构

➤ 前馈神经网络

Scaled Dot-Product Attention



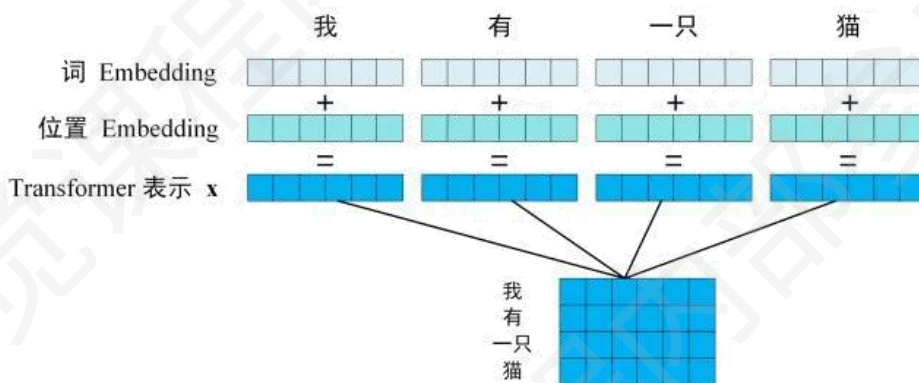
Multi-Head Attention



规格严格 功夫到家

Transformer - 输入

- Word Embedding
- Position Embedding (全局结构，需要表示词的位置)



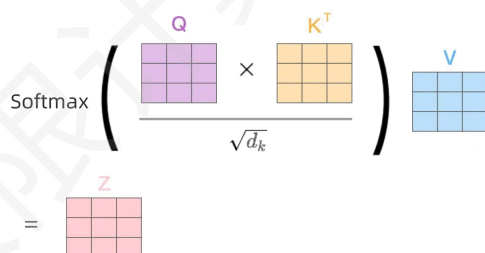
规格严格 功夫到家

Transformer - 自注意力

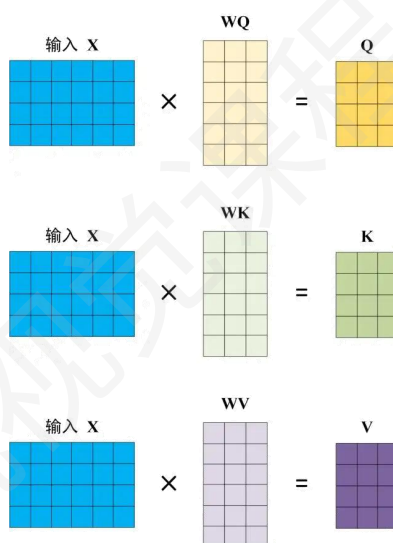
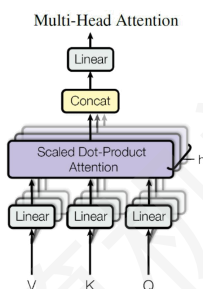
- Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k 是 Q, K 矩阵的列数，即向量维度

$$\text{Softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$


The diagram shows the matrix multiplication of Q (purple) and K^T (orange) to produce Z (pink).



规格严格 功夫到家

Transformer - 编码器-解码器结构

➤ 编码器与解码器

➤ 编码器

➤ 可以看到完整的信息

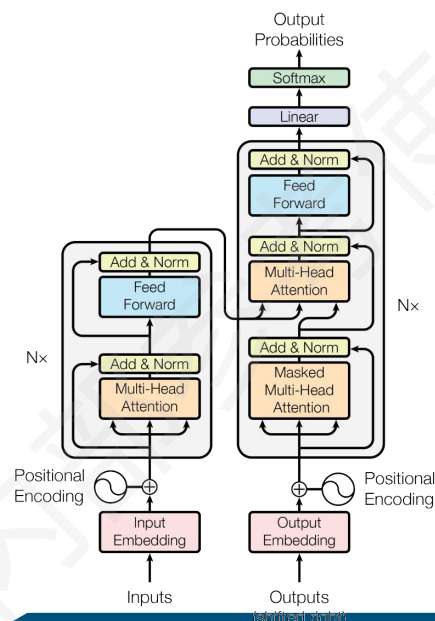
➤ 解码器

➤ 只能看到前面的信息

$$\text{Softmax} \left(\frac{\left(\begin{matrix} \text{Q} \\ \text{K}^T \end{matrix} \right) \times \begin{matrix} \text{Mask} \end{matrix}}{\sqrt{d_k}} \right) \otimes \begin{matrix} \text{V} \end{matrix}$$

按位乘法

$$= \begin{matrix} \text{Z} \end{matrix}$$



规格严格 功夫到家

代表性Transformer模型

➤ Vision Transformer

➤ ViT (大规模预训练+ImageNet-1k微调)

➤ DeiT (ImageNet-1k直接训练)

➤ PVT (引入特征金字塔)

➤ Transformer+卷积

➤ LocalViT (FFN替换为卷积)

➤ SwinTransformer (交错窗口)

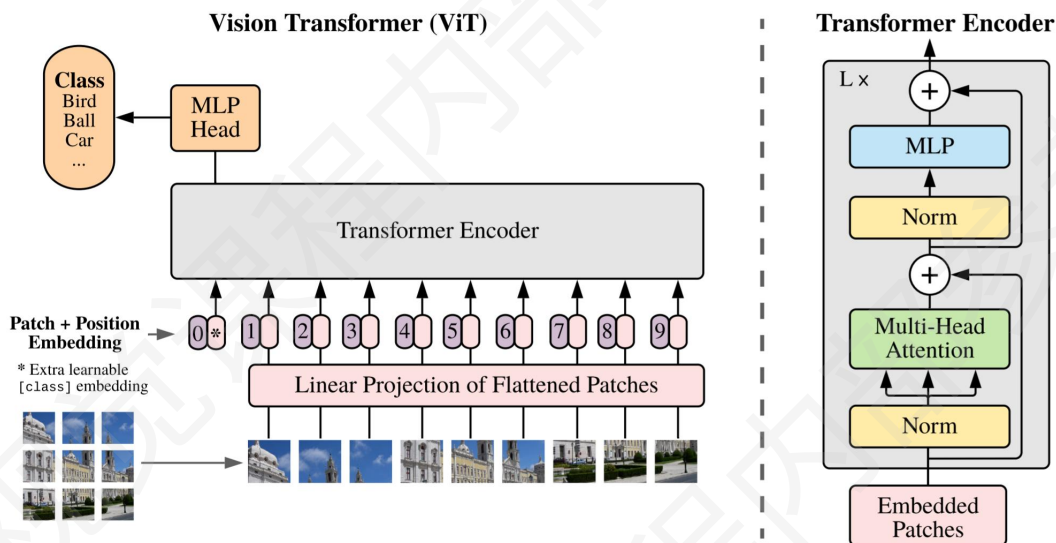
➤ Transformer→卷积

➤ ConvNeXt

➤ RepLKNet

规格严格 功夫到家

Vision Transformer (ViT)



规格严格 功夫到家

DeiT

➤ 对优化算法、超参数、数据增广策略进行枚举搜索

Ablation on ↓	Pre-training	Fine-tuning	Rand-Augment	AutoAug	Mixup	CutMix	Erasing	Stoch. Depth	Repeated Aug.	Dropout	Exp. Moving Avg.	top-1 accuracy	
												pre-trained 224 ²	fine-tuned 384 ²
none: DeiT-B	adamw	adamw	✓	✗	✓	✓	✓	✓	✓	✗	✗	81.8 ±0.2	83.1 ±0.1
optimizer	SGD	adamw	✓	✗	✓	✓	✓	✓	✓	✗	✗	74.5	77.3
	adamw	SGD	✓	✗	✓	✓	✓	✓	✓	✗	✗	81.8	83.1
data augmentation	adamw	adamw	✗	✗	✓	✓	✓	✓	✓	✗	✗	79.6	80.4
	adamw	adamw	✗	✓	✓	✓	✓	✓	✓	✗	✗	81.2	81.9
	adamw	adamw	✓	✗	✗	✓	✓	✓	✓	✗	✗	78.7	79.8
	adamw	adamw	✓	✗	✗	✗	✓	✓	✓	✗	✗	80.0	80.6
	adamw	adamw	✓	✗	✗	✗	✓	✓	✓	✗	✗	75.8	76.7
regularization	adamw	adamw	✓	✗	✓	✓	✗	✓	✓	✗	✗	4.3*	0.1
	adamw	adamw	✓	✗	✓	✓	✓	✗	✓	✗	✗	3.4*	0.1
	adamw	adamw	✓	✗	✓	✓	✓	✗	✗	✗	✗	76.5	77.4
	adamw	adamw	✓	✗	✓	✓	✓	✓	✓	✓	✗	81.3	83.1
	adamw	adamw	✓	✗	✓	✓	✓	✓	✓	✗	✓	81.9	83.1

规格严格 功夫到家

DeiT

➤ AdamW + 强正则 + 强增广 + 更多训练迭代

Methods	ViT-B [15]	DeiT-B
Epochs	300	300
Batch size	4096	1024
Optimizer	AdamW	AdamW
learning rate	0.003	$0.0005 \times \frac{\text{batchsize}}{512}$
Learning rate decay	cosine	cosine
Weight decay	0.3	0.05
Warmup epochs	3.4	5
Label smoothing ϵ	$\times$	0.1
Dropout	0.1	$\times$
Stoch. Depth	$\times$	0.1
Repeated Aug	$\times$	$\checkmark$
Gradient Clip.	$\checkmark$	$\times$
Rand Augment	$\times$	9/0.5
Mixup prob.	$\times$	0.8
Cutmix prob.	$\times$	1.0
Erasing prob.	$\times$	0.25

Table 9: Ingredients and hyper-parameters for our method and ViT-B.

规格严格 功夫到家

Pyramid Vision Transformer

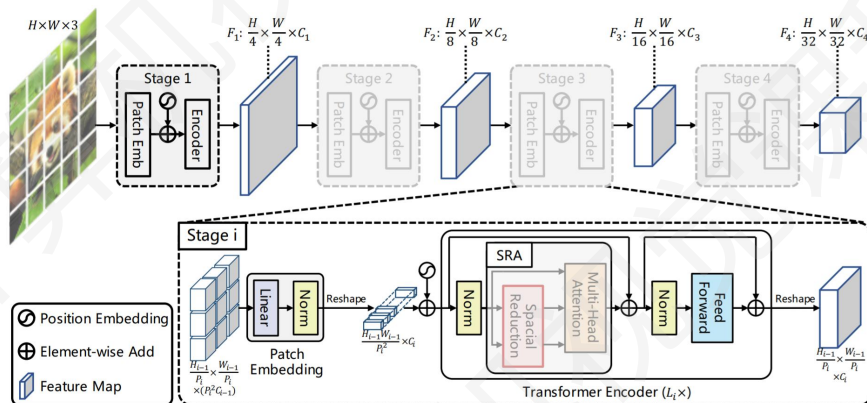


Figure 3: Overall architecture of Pyramid Vision Transformer (PVT). The entire model is divided into four stages, each of which is comprised of a patch embedding layer and a L_i -layer Transformer encoder. Following a pyramid structure, the output resolution of the four stages progressively shrinks from high (4-stride) to low (32-stride).

Early Convolutions Help Transformers See Better

model	flops (B)	params (M)	acts (M)	time (min)	batch size	epochs	IN 21k
ResNet-50	4.1	25.6	11.3	3.4	2048	22.5 21.2 20.7	21.6
ResNet-101	7.8	44.5	16.4	5.5	2048	20.3 19.1 18.5	19.2
ResNet-152	11.5	60.2	22.8	7.7	2048	19.5 18.4 17.7	18.2
ResNet-200	15.0	64.7	32.3	10.7	1024	19.5 18.3 17.6	17.7
RegNetY-1GF	1.0	9.6	6.2	3.1	2048	23.2 22.2 21.5	-
RegNetY-4GF	4.1	22.4	14.5	7.6	2048	19.4 18.3 17.9	18.4
RegNetY-16GF	15.5	72.3	30.7	17.9	1024	17.1 16.4 16.3	15.6
RegNetY-32GF	31.1	128.6	46.2	35.1	512	16.2 15.9 15.9	15.0
RegNetZ-1GF	1.0	11.0	8.8	4.2	2048	20.8 20.2 19.6	-
RegNetZ-4GF	4.0	28.1	24.3	12.9	1024	17.4 16.9 16.6	-
RegNetZ-16GF	16.0	95.3	51.3	32.0	512	16.0 15.9 15.9	-
RegNetZ-32GF	32.0	175.1	79.6	55.3	256	16.3 16.2 16.1	-

model	flops (B)	params (M)	acts (M)	time (min)	batch size	epochs	IN 21k
EffNet-B2	1.0	9.1	13.8	5.9	2048	21.4 20.5 19.9	-
EffNet-B4	4.4	19.3	49.5	19.4	512	18.5 17.8 17.5	-
EffNet-B5	10.3	30.4	98.9	41.7	256	17.3 17.0 17.0	-
ViT _P -1GF	1.1	4.8	5.5	2.6	2048	33.2 29.7 27.7	-
ViT _P -4GF	3.9	18.5	11.1	3.8	2048	23.3 20.8 19.6	20.6
ViT _P -18GF	17.5	86.6	24.0	11.5	1024	19.9 18.4 17.9	16.4
ViT _P -36GF	35.9	178.4	37.3	18.8	512	19.9 18.8 18.2	15.1
ViT _C -1GF	1.1	4.6	5.7	2.7	2048	28.6 26.1 24.7	-
ViT _C -4GF	4.0	17.8	11.3	3.9	2048	20.9 19.2 18.6	18.8
ViT _C -18GF	17.7	81.6	24.1	11.4	1024	18.4 17.5 17.0	15.1
ViT _C -36GF	35.0	167.8	36.7	18.6	512	18.3 17.6 16.8	14.2

Table 2: **Peak performance (grouped by model family)**: Model complexity and validation top-1 error at 100, 200, and 400 epoch schedules on ImageNet-1k, and the top-1 error after pretraining on ImageNet-21k (IN 21k) and fine-tuning on ImageNet-1k. This table serves as reference for the results shown in Figure 6. Blue numbers: best model trainable under 20 minutes per ImageNet-1k epoch. Batch sizes and training times are reported normalized to 8 32GB Volta GPUs (see Appendix).

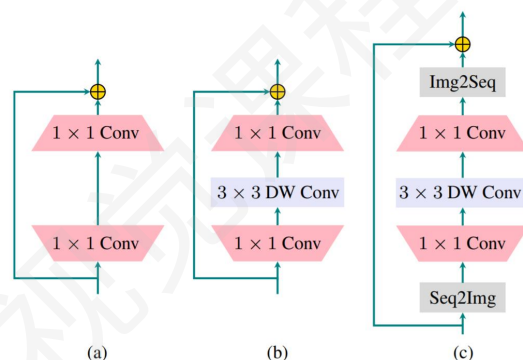
Early Convolutions Help Transformers See Better, Arxiv 2021

规格严格 功夫到家

LocalViT

➤ 将FFN替换成depthwise conv

Network	γ	DW	Params (M)	FLOPs (G)	Top-1 Acc. (%)
DeiT-T [41]	4	No	5.7	1.3	72.2
LocalViT-T	4	No	5.7	1.3	72.5 (0.3 $\uparrow$)
LocalViT-T*	4	Yes	5.8	1.3	73.7 (1.5 $\uparrow$)
DeiT-T [41]	6	No	7.5	1.6	73.1 $\dagger$
LocalViT-T	6	No	7.5	1.6	74.3 (1.2 $\uparrow$)
LocalViT-T*	6	Yes	7.7	1.6	76.1 (3.0 $\uparrow$)



LocalViT: Bringing Locality to Vision Transformers, Arxiv 2021.

规格严格 功夫到家

Swin Transformer

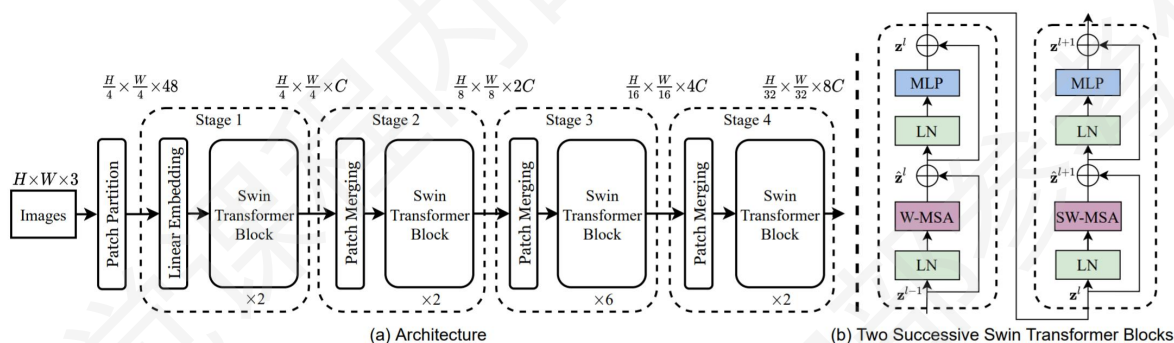


Figure 3. (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks (notation presented with Eq. (3)). W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

Swin Transformer: Hierarchical Vision Transformer using Shifted Windows, ICCV 2021

规格严格 功夫到家

Swin Transformer

交替窗口

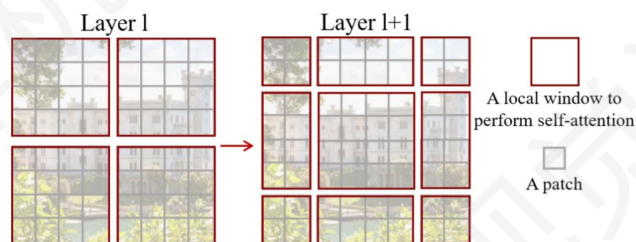


Figure 2. An illustration of the *shifted window* approach for computing self-attention in the proposed Swin Transformer architecture. In layer l (left), a regular window partitioning scheme is adopted, and self-attention is computed within each window. In the next layer $l + 1$ (right), the window partitioning is shifted, resulting in new windows. The self-attention computation in the new windows crosses the boundaries of the previous windows in layer l , providing connections among them.

规格严格 功夫到家

性能对比

(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	320 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
MLP-Mixer-B/16 [61]	224 ²	59M	12.7G	-	76.4
ResMLP-S24 [62]	224 ²	30M	6.0G	715	79.4
ResMLP-B24 [62]	224 ²	116M	23.0G	231	81.0
Swin-T/D24 (Transformer)	256 ²	28M	5.9G	563	81.6
Swin-Mixer-T/D24	256 ²	20M	4.0G	807	79.4
Swin-Mixer-T/D12	256 ²	21M	4.0G	792	79.6
Swin-Mixer-T/D6	256 ²	23M	4.0G	766	79.7
Swin-Mixer-B/D24 (no shift)	224 ²	61M	10.4G	409	80.3
Swin-Mixer-B/D24	224 ²	61M	10.4G	409	81.3

Table 10. Performance of Swin MLP-Mixer on ImageNet-1K classification. D indicates the number of channels per head. Throughput is measured using the GitHub repository of [68] and a V100 GPU, following [63].

规格严格 功夫到家

Vision Transformers -> CNN

- 大卷积核
- Layer Norm
- 激活函数: GELU
- 优化器: AdamW、LAMB
- ConvNeXts compete favorably with Transformers in terms of accuracy and scalability

A ConvNet for the 2020s, Arxiv 2022 (Facebook)

Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs , Arxiv 2022 (旷视)

规格严格 功夫到家

ConvNeXt: 训练技巧

➤ ResNet-50/200

➤ 90 -> 300 epochs

➤ AdamW

➤ 数据增广

➤ 正则化

➤ 76.1% -> 78.8% (ResNet-

(pre-)training config	ConvNeXt-T/S/B/L ImageNet-1K 224 ²	ConvNeXt-B/L/XL ImageNet-22K 224 ²
optimizer	AdamW	AdamW
base learning rate	4e-3	4e-3
weight decay	0.05	0.05
optimizer momentum	$\beta_1, \beta_2=0.9, 0.999$	$\beta_1, \beta_2=0.9, 0.999$
batch size	4096	4096
training epochs	300	90
learning rate schedule	cosine decay	cosine decay
warmup epochs	20	5
warmup schedule	linear	linear
layer-wise lr decay [6, 10]	None	None
randaugment [12]	(9, 0.5)	(9, 0.5)
label smoothing [65]	0.1	0.1
mixup [85]	0.8	0.8
cutmix [84]	1.0	1.0
stochastic depth [34]	0.1/0.4/0.5/0.5	0.1/0.1/0.2
layer scale [69]	1e-6	1e-6
gradient clip	None	None
exp. mov. avg. (EMA) [48]	0.9999	None

A ConvNet for the 2020s, Arxiv 2022

规格严格 功夫到家

ConvNeXt: 宏观设计

➤ 网络模块数

➤ (3, 4, 6, 3) -> (3, 3, 9, s3) (ResNet-50)

➤ 78.8% -> 79.4%

➤ Patchify

➤ 7x7卷积(stride 2) + max pooling → 4x4卷积 (stride 4)

➤ 79.4% -> 79.5%

规格严格 功夫到家

ConvNeXt: ResNeXt思想与反瓶颈结构

➤ ResNeXtify

- 深度可分离卷积
- 增加网络宽度
- -> 80.5%

➤ Inverted Bottleneck

- 80.5% to 80.6%

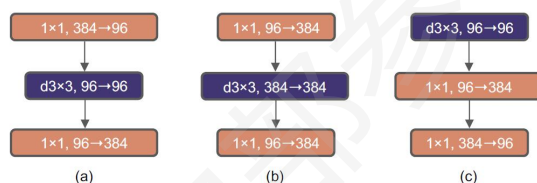


Figure 3. **Block modifications and resulted specifications.** (a) is a ResNeXt block; in (b) we create an inverted bottleneck block and in (c) the position of the spatial depthwise conv layer is moved up.

规格严格 功夫到家

ConvNeXt: Large Kernel Sizes and Micro Design

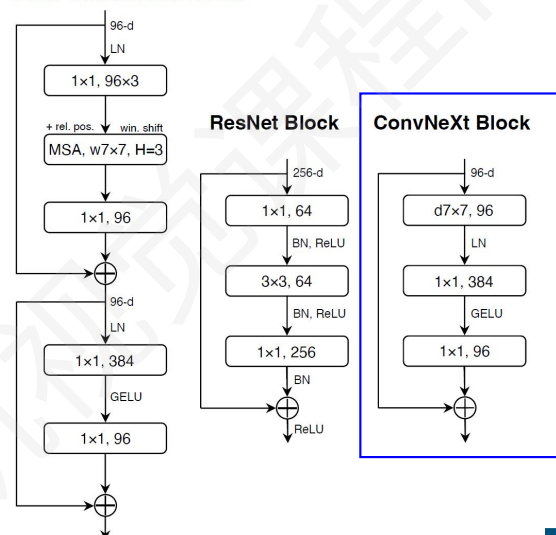
➤ 大卷积核

- 3x3 -> 7x7
- -> 80.6%

➤ 微观设计

- ReLU -> GELU: 80.6%
- 每个模块一个GELU: 81.3%
- 删除两个BN, 只在1 x 1卷积前保留一个: 81.4%
- BN -> LN: 81.5%
- 将空间下采样与通道变换拆分: 82.0%

Swin Transformer Block



规格严格 功夫到家

ConvNeXt

model	image size	#param.	FLOPs	throughput (image / s)	IN-1K top-1 acc.
ImageNet-1K trained models					
• RegNetY-4G [51]	224 ²	21M	4.0G	1156.7	80.0
• RegNetY-8G [51]	224 ²	39M	8.0G	591.6	81.7
• RegNetY-16G [51]	224 ²	84M	16.0G	334.7	82.9
• EffNet-B3 [67]	300 ²	12M	1.8G	732.1	81.6
• EffNet-B4 [67]	380 ²	19M	4.2G	349.4	82.9
• EffNet-B5 [67]	456 ²	30M	9.9G	169.1	83.6
• EffNet-B6 [67]	528 ²	43M	19.0G	96.9	84.0
• EffNet-B7 [67]	600 ²	66M	37.0G	55.1	84.3
○ DeiT-S [68]	224 ²	22M	4.6G	978.5	79.8
○ DeiT-B [68]	224 ²	87M	17.6G	302.1	81.8
○ Swin-T	224 ²	28M	4.5G	757.9	81.3
• ConvNeXt-T	224 ²	29M	4.5G	774.7	82.1
○ Swin-S	224 ²	50M	8.7G	436.7	83.0
• ConvNeXt-S	224 ²	50M	8.7G	447.1	83.1
○ Swin-B	224 ²	88M	15.4G	286.6	83.5
• ConvNeXt-B	224 ²	89M	15.4G	292.1	83.8
○ Swin-B	384 ²	88M	47.1G	85.1	84.5
• ConvNeXt-B	384 ²	89M	45.0G	95.7	85.1
• ConvNeXt-L	224 ²	198M	34.4G	146.8	84.3
• ConvNeXt-L	384 ²	198M	101.0G	50.4	85.5

ImageNet-22K pre-trained models

• R-101x3 [36]	384 ²	388M	204.6G	-	84.4
• R-152x4 [36]	480 ²	937M	840.5G	-	85.4
○ ViT-B/16 [18]	384 ²	87M	55.5G	93.1	84.0
○ ViT-L/16 [18]	384 ²	305M	191.1G	28.5	85.2
○ Swin-B	224 ²	88M	15.4G	286.6	85.2
• ConvNeXt-B	224 ²	89M	15.4G	292.1	85.8
○ Swin-B	384 ²	88M	47.0G	85.1	86.4
• ConvNeXt-B	384 ²	89M	45.1G	95.7	86.8
○ Swin-L	224 ²	197M	34.5G	145.0	86.3
• ConvNeXt-L	224 ²	198M	34.4G	146.8	86.6
○ Swin-L	384 ²	197M	103.9G	46.0	87.3
• ConvNeXt-L	384 ²	198M	101.0G	50.4	87.5
• ConvNeXt-XL	224 ²	350M	60.9G	89.3	87.0
• ConvNeXt-XL	384 ²	350M	179.0G	30.2	87.8

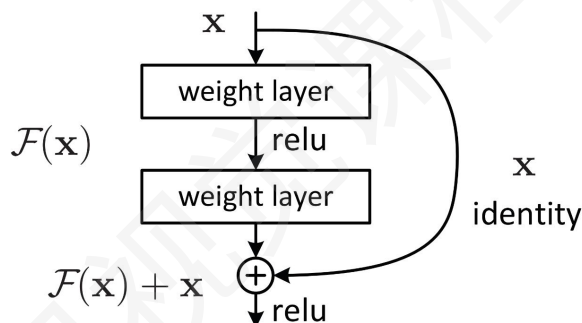
规格严格 功夫到家

RepLKNet: 重参数化

➤ 如果 $F(x) = x \otimes w$, w 为 3×3 卷积

➤ $w' = w + [0, 0, 0; 0, 1, 0; 0, 0, 0]$

➤ $F(x) + x = x \otimes w'$

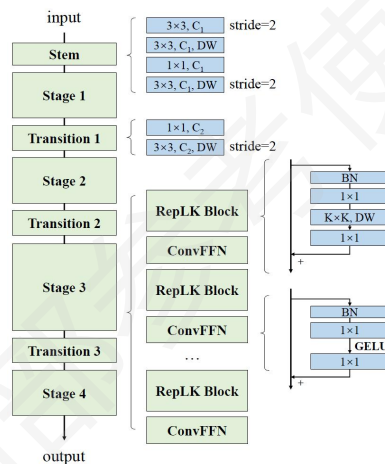
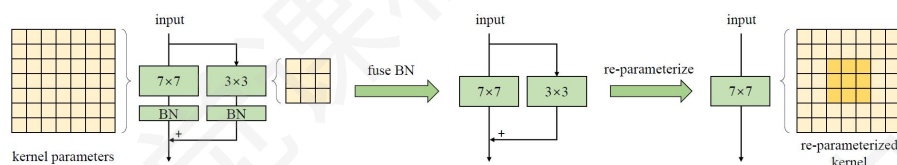


规格严格 功夫到家

RepLKNet

➤ 重参数化大卷积核深度可分离卷积

- 大卷积核深度可分离卷积
- 恒等映射分支
- 重参数化



Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs, Arxiv 2022

規格嚴格 功夫到家

RepLKNet

Model	Input resolution	Top-1 acc	Params (M)	FLOPs (G)	Throughput examples/s
RepLKNet-31B	224×224	83.5	79	15.3	295.5
Swin-B	224×224	83.5	88	15.4	226.2
RepLKNet-31B	384×384	84.8	79	45.1	97.0
Swin-B	384×384	84.5	88	47.0	67.9
RepLKNet-31B[‡]	224×224	85.2	-	-	-
Swin-B [‡]	224×224	85.2	-	-	-
RepLKNet-31B[‡]	384×384	86.0	-	-	-
Swin-B [‡]	384×384	86.4	-	-	-
RepLKNet-31L[‡]	384×384	86.6	172	96.0	50.2
Swin-L [‡]	384×384	87.3	197	103.9	36.2
RepLKNet-XL[◇]	320×320	87.8	335	128.7	39.1

規格嚴格 功夫到家

本节内容 - 视觉神经网络与典型训练范式

➤ 视觉Transformer

➤ 神经网络的典型训练范式

➤ 典型深度学习框架

規格嚴格 功夫到家

学习范式

- 监督学习
- 无监督学习
- 半监督学习
- 弱监督学习
- 主动学习
- 多任务学习
- 迁移学习(域适应/域泛化)
- 元学习
- 在线学习
- 增量学习/持续学习
- 集成学习
- 联邦学习
- 对比学习
- 课程学习

規格嚴格 功夫到家

监督学习

给定数据集 $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ ，学习 $f: \mathcal{X} \rightarrow \mathcal{Y}$ ，最小化

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i), y_i)$$

- 适用场景：标签充足、分布稳定（如图像分类、情感分析）
- 优点：精度高、理论成熟
- 缺点：依赖大规模人工标注，泛化依赖数据分布

规格严格 功夫到家

无监督学习

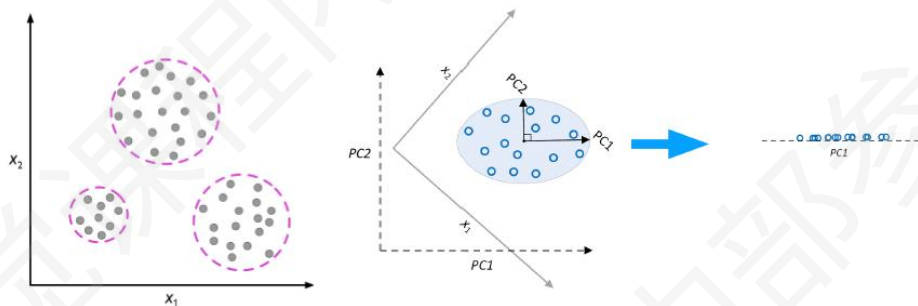
无标签 $\{\mathbf{x}_i\}_{i=1}^N$ ，优化目标如重构、聚类或对比损失

$$\mathcal{L}_{\text{SSL}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

- 适用场景：无标签数据丰富（如聚类、表示学习、数据压缩）
- 优点：无需人工标签，可挖掘数据内在结构
- 缺点：评估困难，下游任务需调优

规格严格 功夫到家

无监督学习



规格严格 功夫到家

半监督学习

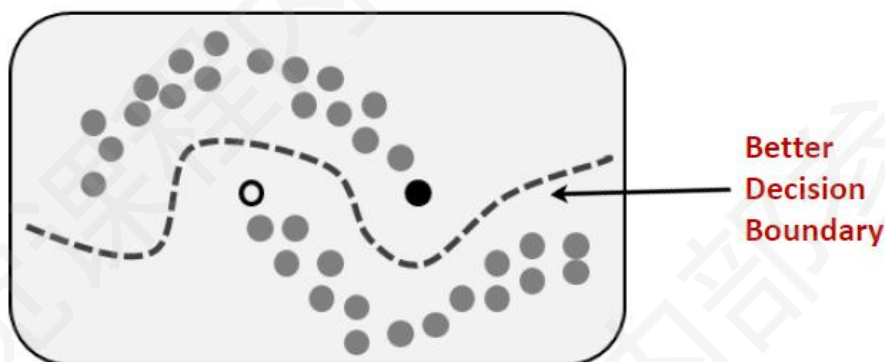
少量有标签 $\mathcal{D}_l$ ，大量无标签 $\mathcal{D}_u$

$$\mathcal{L} = \mathcal{L}_{\text{sup}}(\mathcal{D}_l) + \lambda \mathcal{L}_{\text{unsup}}(\mathcal{D}_u)$$

- 适用场景：标签获取昂贵但大量未标注数据（如医疗影像）
- 优点：减少标注成本，性能接近监督
- 缺点：无标签分布假设错误会伤害性能

规格严格 功夫到家

半监督学习



规格严格 功夫到家

弱监督学习 (广义)

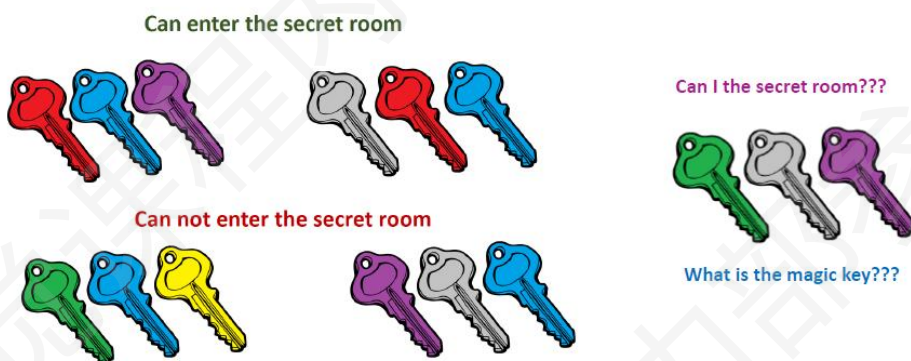
标签不完全、不精确或不准确: $\tilde{y}_i$ 是真实标签的弱近似

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i), \tilde{y}_i) + \text{约束}$$

- 适用场景: 粗粒度标签、噪声标签
- 优点: 利用廉价弱标签
- 缺点: 对标签噪声敏感, 需设计鲁棒损失

规格严格 功夫到家

弱监督学习(广义)



规格严格 功夫到家

主动学习

在未标注池中按查询函数 $Q(\mathbf{x})$ 选择最有价值的样本标注:

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in \mathcal{U}} Q(\mathbf{x})$$

- **适用场景:** 标注预算有限 (如药物筛选、罕见病检测)
- **优点:** 用最少标注达到高精度
- **缺点:** 需要迭代交互, 查询策略设计复杂

规格严格 功夫到家



主动学习



規格嚴格 功夫到家



主动学习



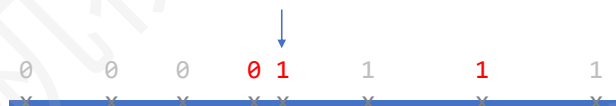
規格嚴格 功夫到家

主动学习



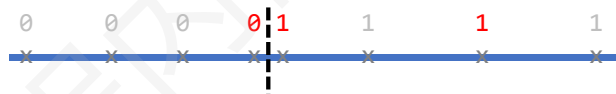
規格嚴格 功夫到家

主动学习



規格嚴格 功夫到家

主动学习



规格严格 功夫到家

多任务学习

共享表示 $\phi(\mathbf{x})$ ，多个任务损失加权
 (区别于弱监督学习中的Multi-Instance Learning)

$$\mathcal{L}_{\text{MTL}} = \sum_{t=1}^T \alpha_t \mathcal{L}_t(\mathbf{W}_t^\top \phi(\mathbf{x}), y^{(t)})$$

- 适用场景：标注预算有限（如药物筛选、罕见病检测）
- 优点：用最少标注达到高精度
- 缺点：需要迭代交互，查询策略设计复杂

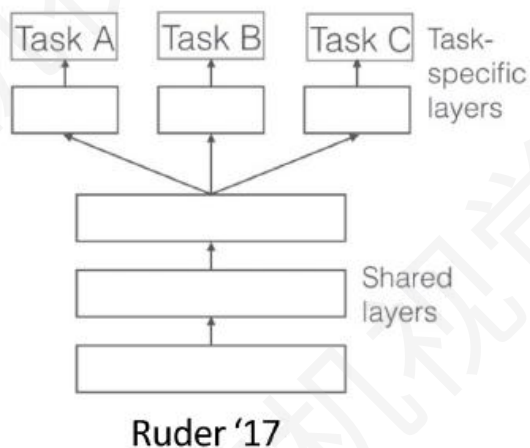
规格严格 功夫到家

多任务学习



規格嚴格 功夫到家

多任务学习



規格嚴格 功夫到家

迁移学习

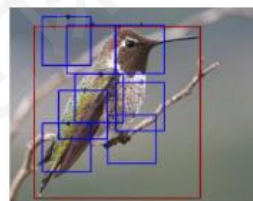
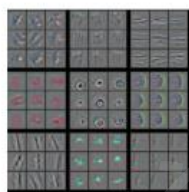
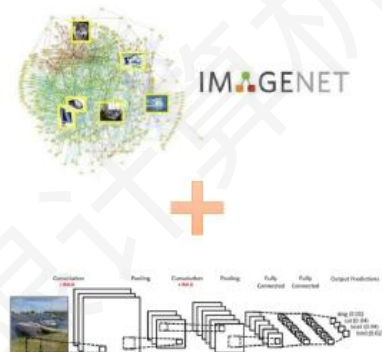
源域 $\mathcal{D}_s$ 上学习知识，迁移到目标域 $\mathcal{D}_t$ (通常 $P_s \neq P_t$):

$$\theta^* = \arg \min_{\theta} \mathcal{L}_t(\theta; \mathcal{D}_t) \quad \text{s.t. } \theta \text{ 初始化自 } \theta_s$$

- 适用场景：目标任务数据少（BERT微调、ImageNet预训练）
- 优点：大幅降低数据需求
- 缺点：负迁移风险，源域与目标域差异大时无效

规格严格 功夫到家

迁移学习



规格严格 功夫到家

元学习

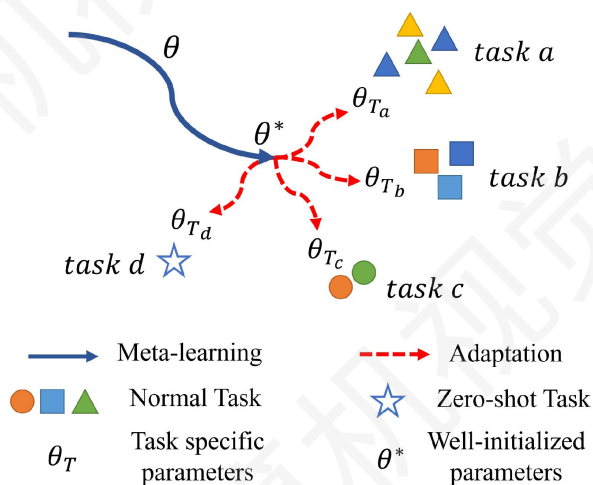
在多个任务 $\{\mathcal{T}_k\}$ 上优化元参数 θ :

$$\theta^* = \arg \min_{\theta} \sum_k \mathcal{L}_k^{\text{test}}(\theta - \alpha \nabla \mathcal{L}_k^{\text{train}}(\theta))$$

- 适用场景：少样本学习（如新类别分类、机器人快速适应）
- 优点：极快适应新任务
- 缺点：元训练需要大量任务，易过拟合

规格严格 功夫到家

元学习



规格严格 功夫到家

在线学习

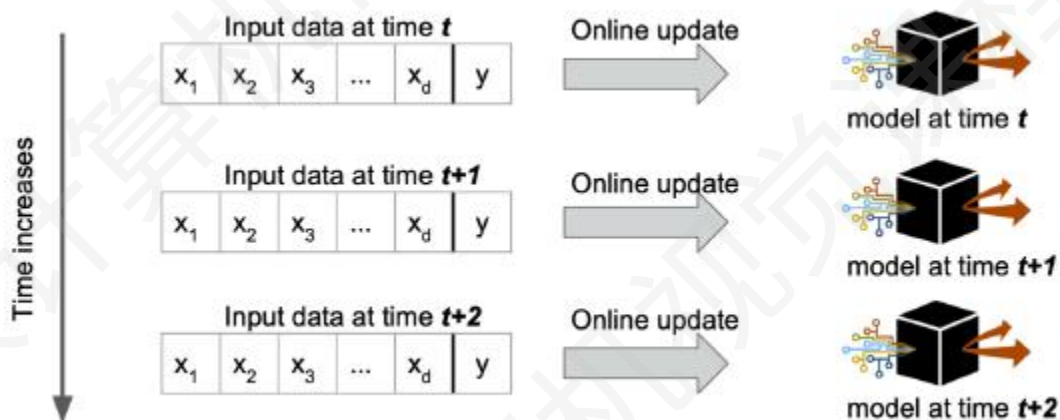
在时间步 t 收到样本 $(\mathbf{x}_t, y_t)$, 更新模型:

$$\theta_{t+1} = \theta_t - \eta_t \nabla \ell(\theta_t; \mathbf{x}_t, y_t)$$

- 适用场景: 数据流、概念漂移 (如推荐系统、股票预测)
- 优点: 实时更新, 内存小
- 缺点: 对噪声敏感, 灾难性遗忘

规格严格 功夫到家

在线学习



规格严格 功夫到家

增量学习/持续学习

设当前模型参数为 θ_{t-1} ，已见数据分布 $P_{1:t-1}$ ，新到来数据批次 $\mathcal{D}_t$ （可能包含新类别 C_{new} ）。增量学习的目标是：

$$\theta_t = \arg \min_{\theta} [\mathcal{L}_{\text{new}}(\theta; \mathcal{D}_t) + \lambda \cdot \mathcal{L}_{\text{old}}(\theta; \mathcal{M}_{1:t-1})]$$

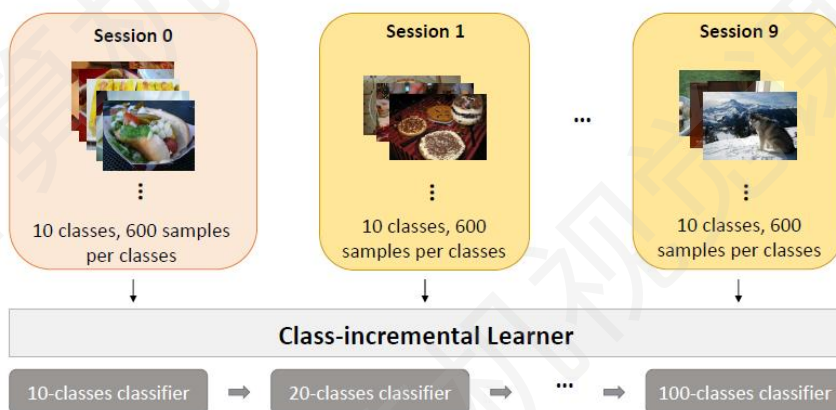
其中 $\mathcal{M}_{1:t-1}$ 是旧知识维持机制（如 正则化项、Replay 等）。

- 适用场景：类别、任务、领域新增（如推荐系统、自动驾驶）
- 优点：能持续吸收新知识，无需从头训练
- 缺点：灾难性遗忘 (Catastrophic Forgetting) 风险高，重放策略可能违反隐私或存储限制

规格严格 功夫到家

增量学习/持续学习

Goal: **Incrementally** learn **new** classes



规格严格 功夫到家

增量学习/持续学习



规格严格 功夫到家

集成学习

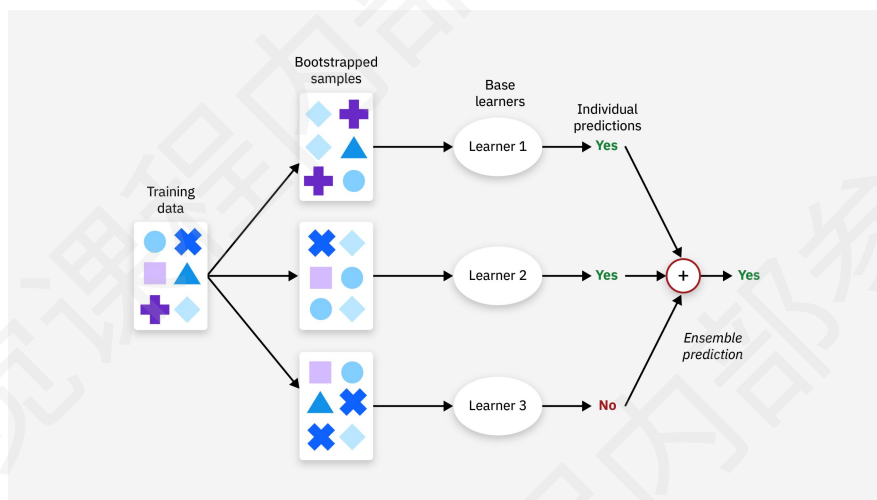
组合 M 个基模型 h_m ，进行预测

$$f_{\text{ens}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M h_m(\mathbf{x})$$

- 适用场景：追求高精度、稳定性（如Kaggle竞赛）
- 优点：降低方差/偏差，鲁棒
- 缺点：计算存储成本高，可解释性差

规格严格 功夫到家

集成学习



規格嚴格 功夫到家

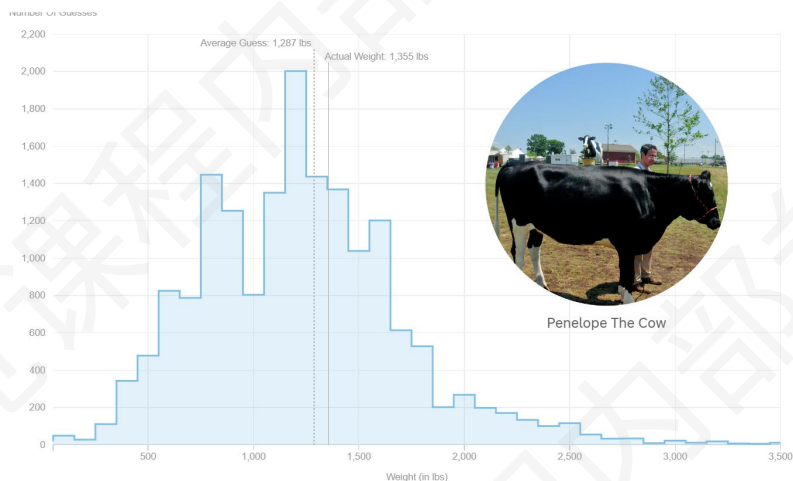
How Much Does This Cow Weigh?

➤ 人：165磅



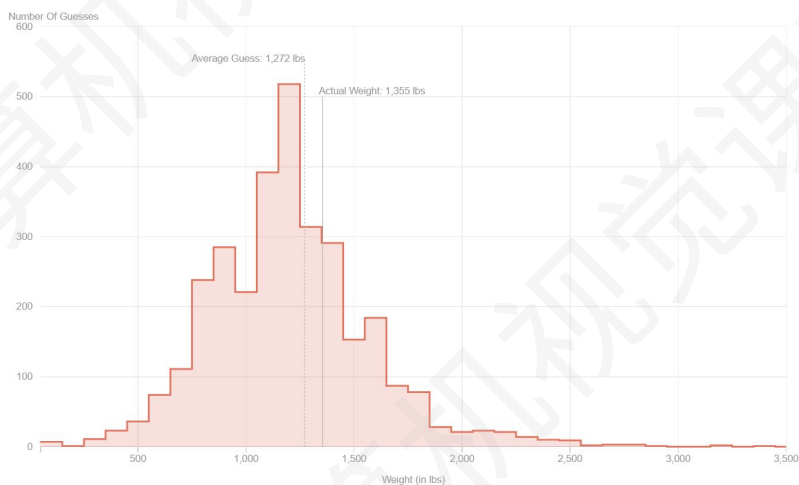
規格嚴格 功夫到家

How Much Does This Cow Weigh? 所有人



規格嚴格 功夫到家

How Much Does This Cow Weigh? 专家



規格嚴格 功夫到家

联邦学习

中心服务器聚合各客户端更新（如FedAvg）：

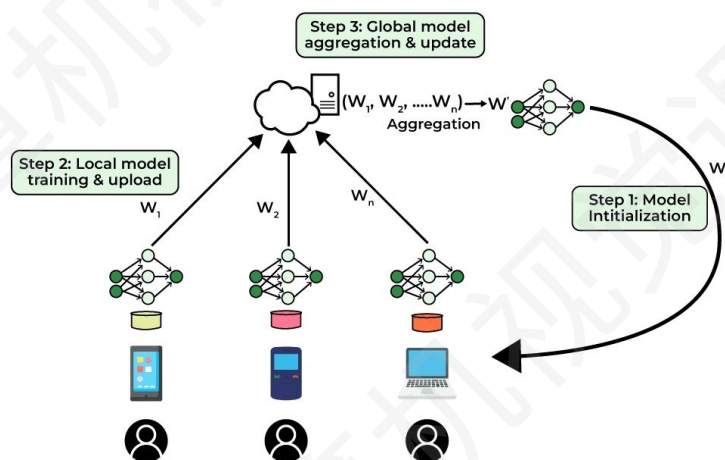
$$\theta_{t+1} = \sum_{k=1}^K \frac{n_k}{n} \theta_{t+1}^{(k)}$$

其中 $\theta_{t+1}^{(k)}$ 在本地数据上训练。

- 适用场景：隐私敏感、数据分散（如医疗、手机输入法）
- 优点：数据不出本地，合规
- 缺点：通信开销大，异质性数据导致性能下降

规格严格 功夫到家

联邦学习



规格严格 功夫到家

对比学习

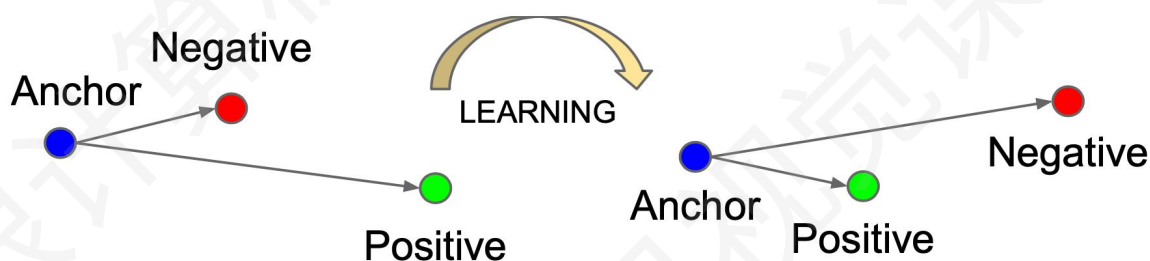
拉近正样本对，推远负样本对：

$$\mathcal{L}_{\text{InfoNCE}} = - \sum_i \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_{i+} / \tau)}{\sum_{j \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}$$

- 适用场景：视觉/语音无监督表示学习（如SimCLR, MoCo）
- 优点：表示质量高，接近监督
- 缺点：需要大批量或动量编码器，负样本设计关键

规格严格 功夫到家

对比学习



规格严格 功夫到家

课程学习

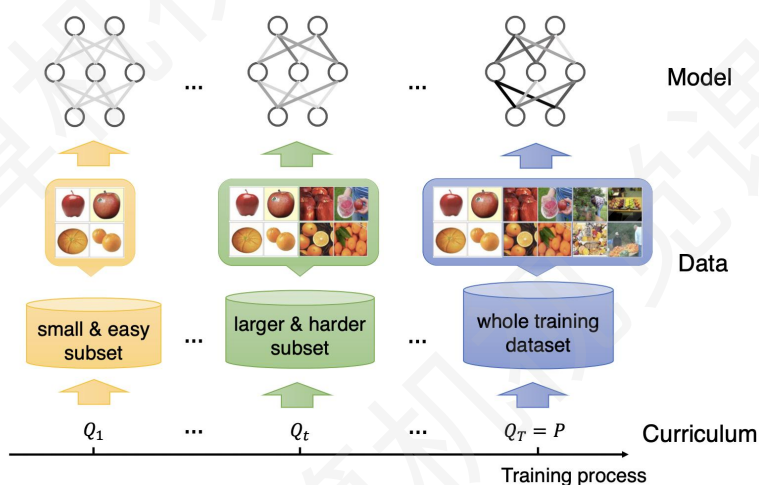
训练时按难度 $d(\mathbf{x})$ 排序样本，逐步增加难度：

$$\mathcal{D}^{(t)} \subset \{\mathbf{x} \mid d(\mathbf{x}) \leq \epsilon_t\}, \quad \epsilon_t \uparrow$$

- 适用场景：困难任务（如神经机器翻译、强化学习探索）
- 优点：加速收敛，提升最终性能
- 缺点：需要定义难度测度，可能引入偏差

规格严格 功夫到家

课程学习



规格严格 功夫到家

本节内容 – 视觉神经网络与典型训练范式

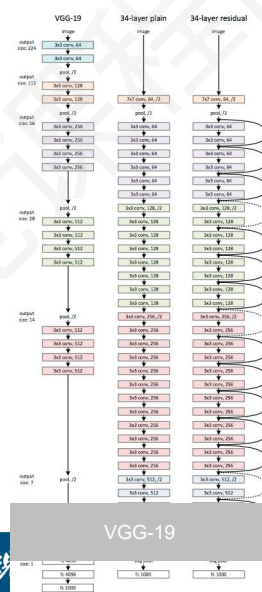
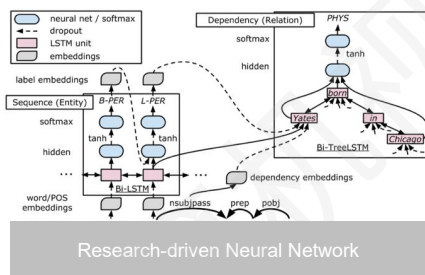
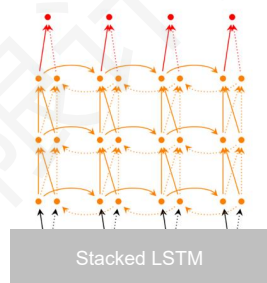
➤ 视觉Transformer

➤ 神经网络的典型训练范式

➤ 典型深度学习框架

规格严格 功夫到家

实现神经网络的挑战：更深、更大、更复杂



利用Numpy实现神经网络？

- 数据准备
- 网络实现
 - 算子实现 (卷积、激活、池化.....)
 - 参数初始化
 - 损失函数实现
- 模型训练
 - 梯度求导
 - 梯度计算
 - 优化算法实现

规格严格 功夫到家

姓名	语言	单位	社会关系	优点	缺点
Caffe	C++/Python /Matlab	BVLC		速度快、模块化 源代码写得十分优雅	灵活性较差，写代码难度高 需要手写C++/CUDA正向反向
Caffe2	C++/Python	FB	看名字就知道是Caffe的儿子	基于Caffe改进，速度更快，模块化更好	还没流行起来就被并入PyTorch
Keras	Python	GMNA*	坐拥TF/CNTK/Theano /MXNet等几大后台	High-level API简单统一整合多种后端， Google/MS/Nvidia/Amazon支持，借鉴Torch	使用不够灵活，难以实现自定义功能
MatConvNet	Matlab	VLFeat		基于Matlab语言	最新版本为2017年8月发布，灵活性较差
MindSpore	Python	华为		支持国产芯片	算子支持稍差
PyTorch	Python/C++	FB	集Caffe2与Torch于一身	上手容易、代码简单灵活 动态计算图、自动求导	运行效率相比TF略低
TensorFlow	C++/Python /Go/Java/...	Google	亲儿子Tensorboard 可视化利器	大厂出品，更新快，优化好 现已加入动态计算图豪华套餐	入门较难，API繁多 (2.0有精简，但是杂乱无序)
Theano	Python	MILA***		深度学习框架鼻祖之一 集成Numpy，计算稳定性好	2017年9月宣布停止重大更新
Torch7	Lua	FB		速度快(Lua-JIT可达C的80%) 可直接调用C、可移植性好	Lua受众较少，学习成本高
JAX	Python, XLA	Google	Numpy替代品	函数式编程，XLA编译加速	生态不完善，调试难度大

* GMNA: Google, Microsoft, NVIDIA, Amazon AWS

** MAFI: Microsoft, Amazon, Facebook, IBM等

*** MILA: Montréal Institute for Learning Algorithms of Université de Montréal(蒙特利尔大学), Scientific Director: Yoshua Bengio

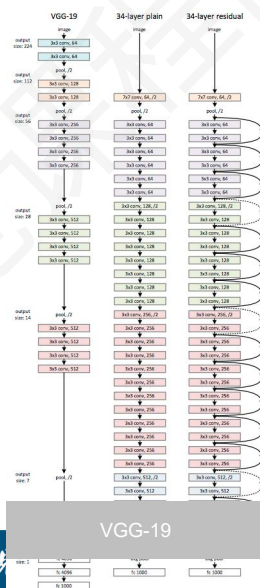
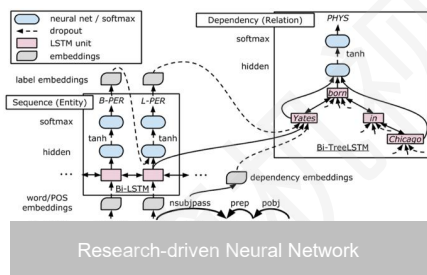
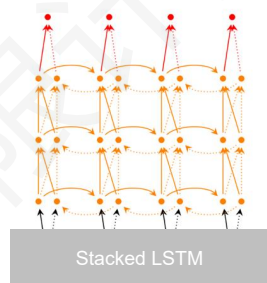
Chainer CNTK Deeplearning4J Dlib H2O Lasagne Leaf MXNet Neon PaddlePaddle PyLearn等如有兴趣请自行探索

规格严格 功夫到家

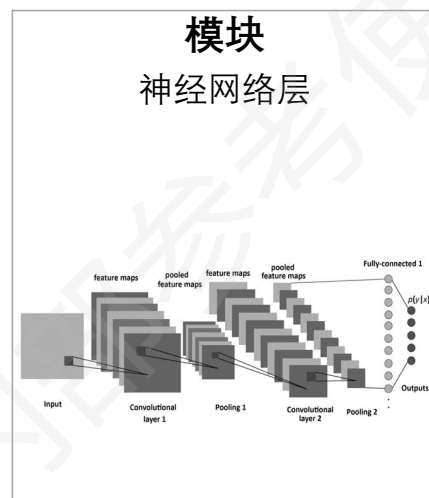
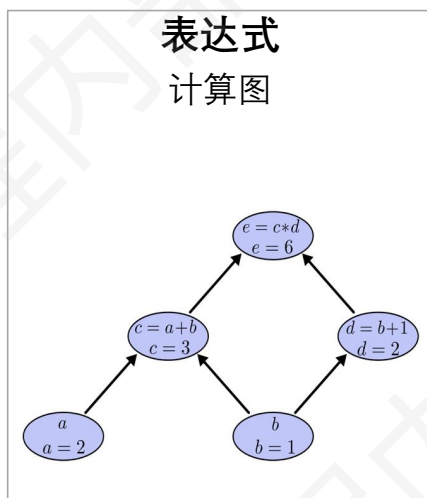
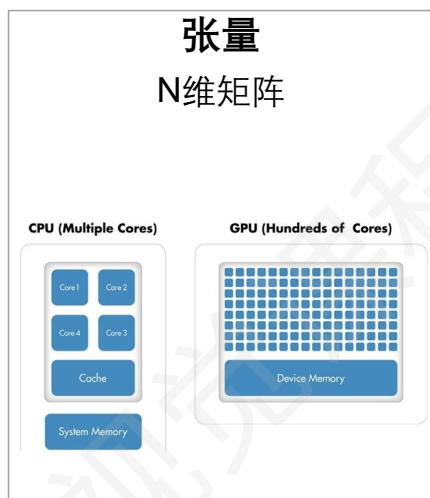
姓名	语言	单位	社会关系	优点	缺点
TensorRT	Python/C++	NVIDIA	可解析ONNX等格式	极致性能，高度量化、生态集成好	仅支持NVIDIA GPU，构建复杂
OpenVINO	Python/C++	Intel	可解析ONNX、TF等模型	Intel硬件表现优异，工具链完善	对非Intel硬件支持非常有限
ONNX	Python/C++等	Microsoft	可对接多种硬件后端	跨平台首选引擎，硬件兼容性强，模型生态好	在特定硬件性能略逊色于厂商专用框架
Anakin	C++	百度	支持Caffe、Paddle等	跨平台性能好（CPU、GPU、ARM）、汇编级优化	社区活跃度低、生态局限
vLLM	Python	UC Berkeley	支持Hugging Face	PagedAttention技术，提升显存利用率，吞吐量高，易用性好	专注LLM场景，主要针对文本生成，不适用于CNN等传统模型
vLLM-Omni	Python	UC Berkeley		首个支持文图音视统一生成的开源框架，资源效率高，性能优异	发布时间短（2025年底），生态不完善
LMDeploy	Python	Shanghai AI Lab	专为LLM设计	解码速度快，量化支持好，部署简单	硬件支持有限（主要支持NVIDIA）、模型兼容性有限

规格严格 功夫到家

计算图(有向无环图)、操作模块化

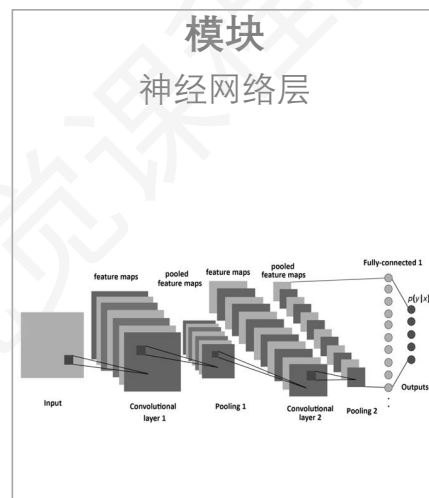
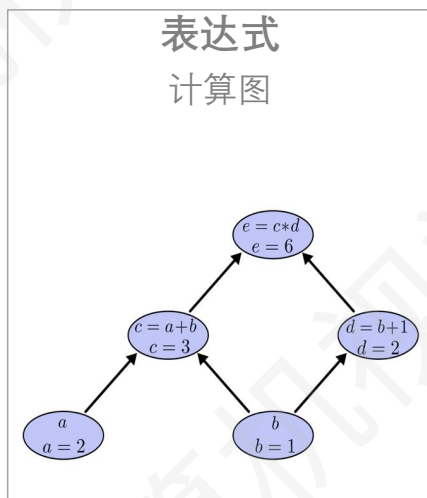
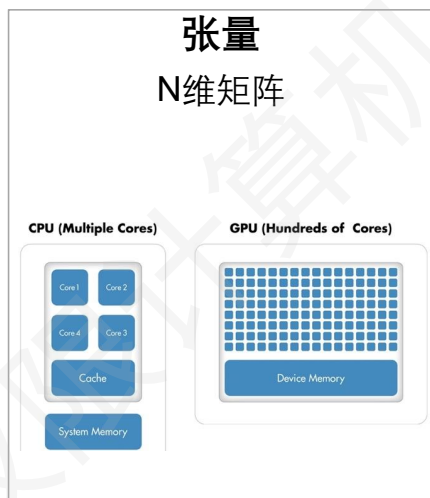


软件工程角度：抽象与复用



规格严格 功夫到家

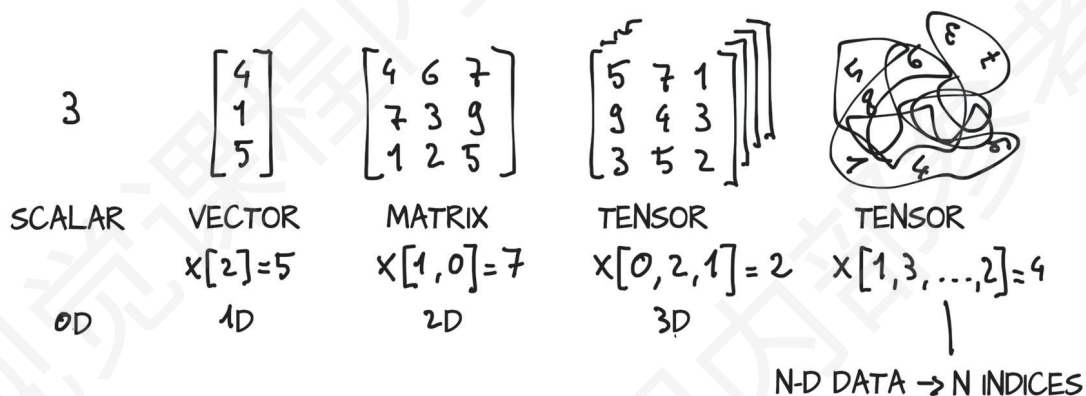
软件工程角度：抽象与复用



规格严格 功夫到家

张量 (Tensor)

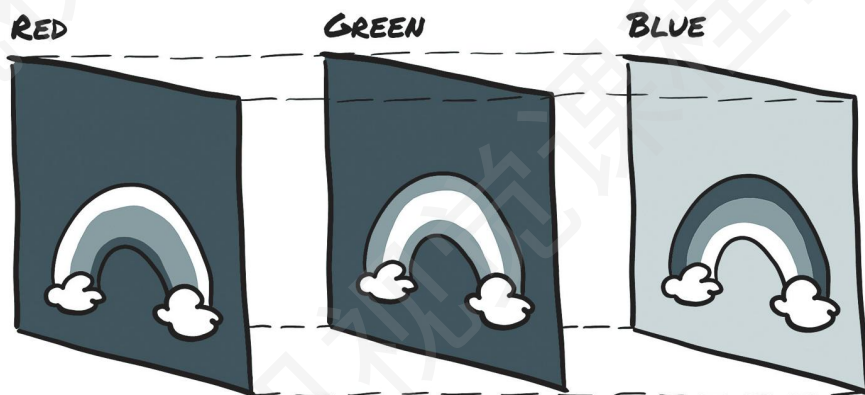
➤ 张量是多维矩阵



规格严格 功夫到家

真实数据的张量化表示

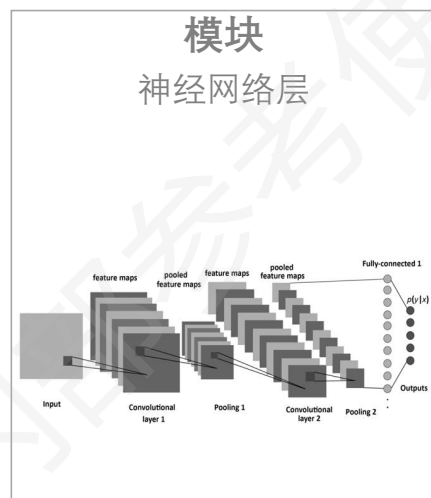
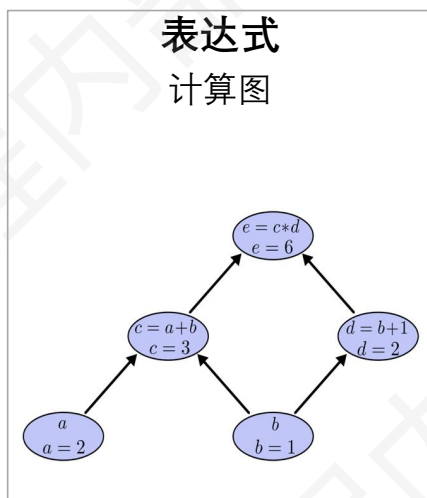
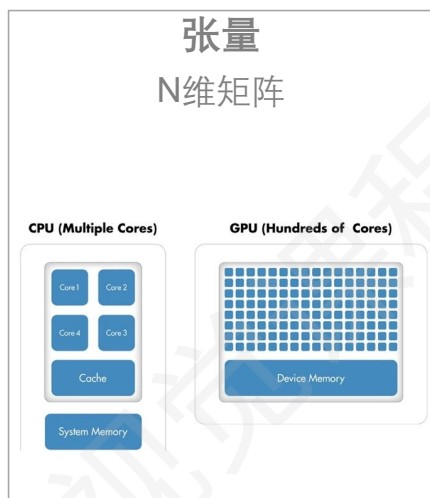
- 表格数据
- 时序数据
- 文本
- 图像



```
batch = torch.zeros(100, 3, 256, 256, dtype=torch.uint8)
```

规格严格 功夫到家

软件工程角度：抽象与复用



规格严格 功夫到家

$e = (a + b) \times (b + 1)$: 前向

```

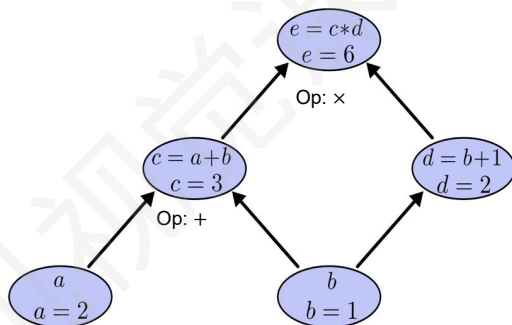
a = torch.tensor([2.], requires_grad=True)
b = torch.tensor([1.], requires_grad=True)

c = a + b
d = b + 1
e = c * d

print(c)
print(d)
print(e)

tensor([3.], grad_fn=<AddBackward0>)
tensor([2.], grad_fn=<AddBackward0>)
tensor([6.], grad_fn=<MulBackward0>)
  
```

$$\frac{\partial e}{\partial a} = b + 1, \frac{\partial e}{\partial b} = 2b + a + 1$$



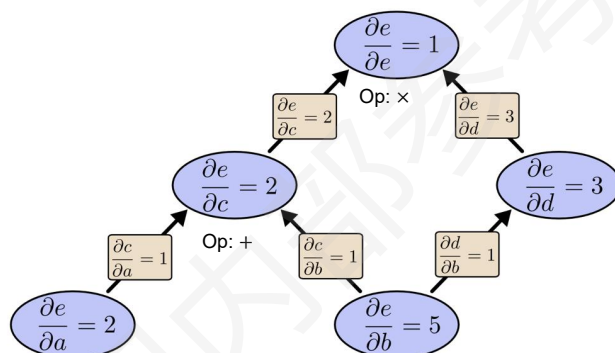
规格严格 功夫到家

$e = (a + b) \times (b + 1)$: 反向

$$\frac{\partial e}{\partial a} = b + 1, \frac{\partial e}{\partial b} = 2b + a + 1$$

```
e.backward()
print(a.grad)
print(b.grad)

tensor([2.])
tensor([5.])
```



规格严格 功夫到家

$e = (a + b) \times (b + 1)$: 反向

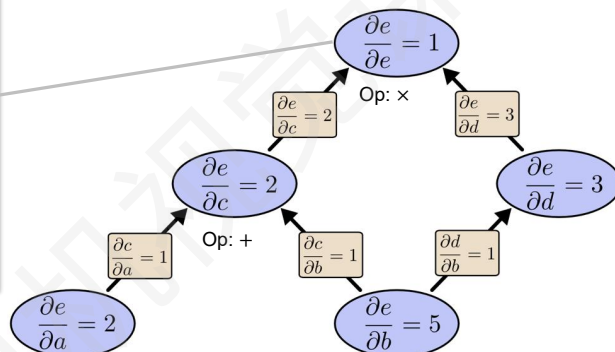
$$\frac{\partial e}{\partial a} = b + 1, \frac{\partial e}{\partial b} = 2b + a + 1$$

```
class MultiplyNode:
    def __init__(self, node1, node2):
        self.node1 = node1
        self.node2 = node2

    def forward(self):
        self.val = self.node1.val * self.node2.val

    def backward(self):
        node1.err = self.err * self.node2.val
        node2.err = self.err * self.node1.val

terminal_node.error = 1
for node in topological_order(cg.nodes):
    if node is not terminal:
        node.backward()
```



规格严格 功夫到家

$$e = (a + b) \times (b + 1)$$



```
import tensorflow as tf

a = tf.placeholder(tf.float32)
b = tf.placeholder(tf.float32)

c = a + b
d = b + 1
e = c * d

grad_a, grad_b = tf.gradients(e, [a, b])

with tf.Session() as sess:
    grad_a, grad_b = sess.run([grad_a, grad_b],
                              feed_dict={a: 2., b: 1.})

    print(grad_a)
    print(grad_b)
```

2.0
5.0



```
import torch
a = torch.tensor([2.], requires_grad=True)
b = torch.tensor([1.], requires_grad=True)

c = a + b
d = b + 1
e = c * d

e.backward()

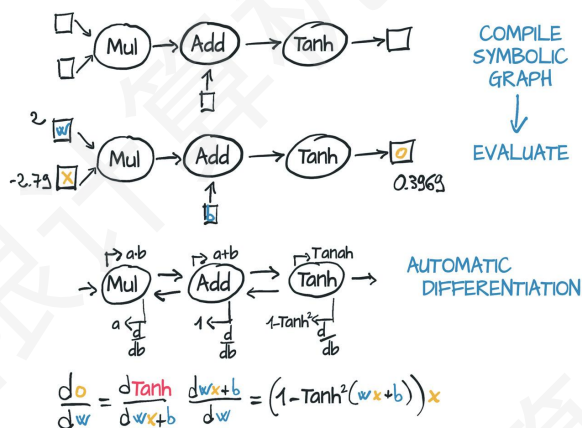
print(a.grad)
print(b.grad)
```

tensor([2.])
tensor([5.])

规格严格 功夫到家

静态图 vs 动态图

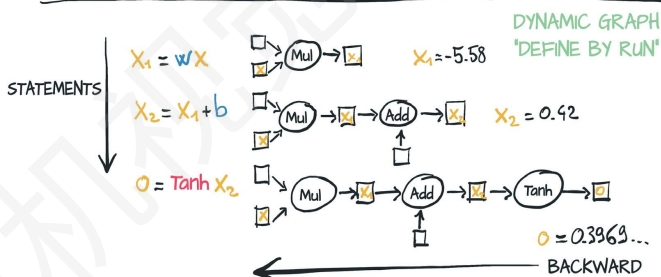
STATIC GRAPH $o = \text{Tanh}(wx + b)$



$o = \text{Tanh}(wx + b)$

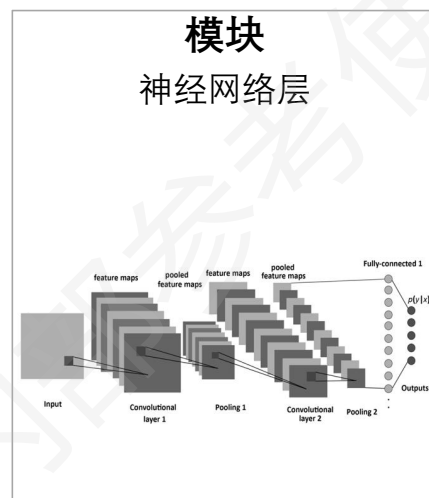
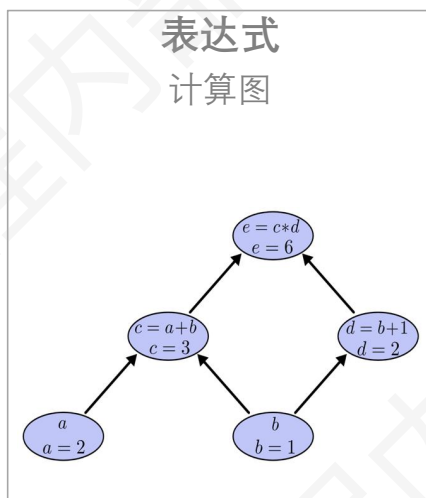
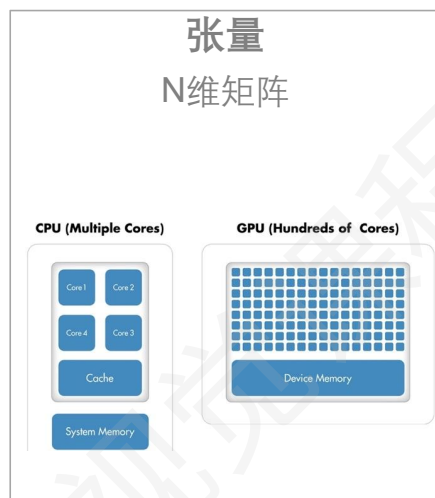
$x = -2.79$
 $x_1 = wx = 2 \times (-2.79) = -5.58$
 $x_2 = x_1 + b = -5.58 + 6 = 0.42$
 $o = \text{Tanh } x_2 = \text{Tanh } 0.42 = 0.3963 \dots$

GREEDY EVALUATION
(NO GRAPH)



规格严格 功夫到家

软件工程角度：抽象与复用



规格严格 功夫到家

PyTorch主要包

包	抽象层	描述
torch	Tensor / Expression	a Tensor library like NumPy, with strong GPU support. When configured with requires_grad, it supports differentiation
torch.nn	Module	a neural networks library deeply integrated with autograd designed for maximum flexibility
torch.optim	Module	an optimization package to be used with torch.nn with standard optimization methods such as SGD, RMSProp, LBFGS, Adam etc.
torch.utils	Module	DataLoader, Dataset and other utility functions for convenience

规格严格 功夫到家

PyTorch APIs

➤ torch

- `>>> import torch`
- `torch.set_num_threads(num)` # *command for limiting threads*
- `torch.set_default_tensor_type(type)` # *e.g., type = torch.float32*
- `torch.zeros/ones/eye/rand/randn/randperm`
全0/全1/对角线元素为1/[0,1)均匀分布/ $N(0,1)$ 高斯分布/[0,n-1]随机排序
- `torch.split/cat/squeeze/unsqueeze/stack`
切分/连接/去除维度/插入维度/堆叠
- `torch.abs/add/sub/mul/div/pow/exp/neg/sqrt/sin/dot`
- `torch.mean/sum/median/max/min/std/var/eq/lt/gt/le/ge/`

規格嚴格 功夫到家

PyTorch APIs

➤ torch.Tensor

- `torch.[Float/Double/Byte/Char/Short/Int/Long]Tensor`
- `torch.cuda.[Float/Double/Half/Byte/Char/Short/Int/Long]Tensor`
- `t.cuda()` / `t.to(device)` / `t.is_cuda()`
- `t.abs()` ↔ `torch.abs(t)`
NOTE: t.fun_() is in-place operation same as t.fun()
- `t.transpose(dim0, dim1)` / `t.permute(*dims)` # `t.permute(2,0,1)`
HWC → CHW
- `t.type_as(t1)`
- `t.view(*shape)` / `t.contiguous()` # *tensor.view 要求内存连续*

規格嚴格 功夫到家

PyTorch APIs

➤ torch.nn

- nn.Module # *base class of all network layers*
 - .cpu|.cuda / .train|.eval / .forward / .parameters / .zero_grad
- nn.Sequential / nn.ModuleList
- nn.Conv?d/ConvTransposed?d/MaxPool?d/AvgPool?d
- nn.ReLU/ELU/LeakyReLU/PReLU/Sigmoid/Tanh/Softplus/Softmax
- nn.BatchNorm?d/SyncBatchNorm/InstanceNorm?d/LayerNorm/GroupNorm/
- nn.RNN/LSTM/GRU
- nn.Linear
- nn.Dropout
- nn.DataParallel

規格嚴格 功夫到家

PyTorch APIs

➤ torch.nn

- nn.L1Loss/MSELoss/BCELoss/CrossEntropyLoss/NLLLoss
- nn.PixelShuffle/nn.Upsample/UpsamplingNearest2d/UpsamplingBilinear2d
- nn.functional # *functions of previously mentioned layers*

➤ torch.optim

- optim.SGD/Adam/...
 - optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
 - optimizer = optim.Adam([
 - {'params': model.base.parameters()},
 - {'params': model.classifier.parameters(), 'lr': 1e-3}
], lr=1e-2, momentum=0.9)

規格嚴格 功夫到家

PyTorch APIs

- torch.utils
 - utils.checkpoint / torch.save|torch.load
 - utils.data
 - .Dataset
 - .DataLoader
 - utils.model_zoo.load_url
 - utils.tensorboard

規格嚴格 功夫到家

PyTorch APIs

- torchvision
 - torchvision.datasets (some are listed below)
 - MNIST / Fashion-MNIST / COCO / LSUN / CIFAR / VOC / Cityscapes / Imagenet-12
 - torchvision.models
 - AlexNet / VGG / ResNet / DenseNet / Inception / GoogleNet / ShuffleNet
 - torchvision.transforms
 - Compose # *Compose a series of transform operations*
 - CenterCrop/RandomCrop/RandomHorizontalFlip/RandomRotation/Resize
 - Normalize/ToPILImage/ToTensor/
 - torchvision.utils.save_image

規格嚴格 功夫到家

PyTorch安装 (<https://pytorch.org/>)

Install PyTorch

Select your preferences and run the install command. Stable represents the most currently tested and supported version of PyTorch. This should be suitable for many users. Preview is available if you want the latest, not fully tested and supported, builds that are generated nightly. Please ensure that you have **met the prerequisites below** (e.g., **numpy**), depending on your package manager. You can also **install previous versions of PyTorch**. Note that LibTorch is only available for C++.

NOTE: Latest Stable PyTorch requires Python 3.10 or later.

PyTorch Build	Stable (2.11.0)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Pip	LibTorch	Source	
Language	Python		C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	ROCm 7.2
Run this Command:	pip3 install torch torchvision			

- 编程语言
 - Python
- 操作系统
 - Linux/Mac/Windows
- 安装方式
 - Pip (推荐anaconda环境)
- 计算平台
 - CUDA/ROCm/CPU

规格严格 功夫到家

PyTorch Resources

- PyTorch Document
 - <https://pytorch.org/docs/stable/index.html> (EN)
 - <https://pytorch-cn.readthedocs.io/zh/latest/> (ZH)
- 矩阵求导
 - 简单教程 <https://github.com/LynnHo/Matrix-Calculus>
 - 求导网站 <http://www.matrixcalculus.org/>
- Python教程
 - 菜鸟教程 <https://www.runoob.com/python/python-tutorial.html>
- PyTorch教程
 - 莫烦系列 <https://morvanzhou.github.io/tutorials/machine-learning/torch/>

规格严格 功夫到家

T h e E n d

THANKS FOR YOUR ATTENTION

規格嚴格 功夫到家